

Stock Movement Prediction

Web Science - COMPSCI5078 | Assessed Coursework Report

Student Name: Vaishnavi Balkrishna Kshirsagar

Student id: 3164851k

1.Introduction

Many factors, such as past price trends, investor sentiment, and external information, affect financial markets. For years, predicting stock prices has been an important topic in financial analysis and machine learning. Traditional models primarily use historical prices to construct technical indicators. However, new data science methods allow researchers to incorporate additional information, such as social media sentiment.

In this report, we created a system to predict whether stock prices will go up or fall. We used both technical market data and fundamental data from Twitter tweets. The dataset used was from the stocknet dataset, which provides historical prices from Yahoo Finance and tweet data associated with a set of US stocks. The workflow involves preprocessing and merging technical stock data with tweets, building neural network models such as LSTM, Bidirectional LSTM (BiLSTM), Random Forest Classifier, and Gradient Boosting (XGBoost), evaluating model performance using various metrics, and exploring advanced features for deeper analysis. By combining market technical indicators with sentiment and engagement signals from Twitter, this assessment aims to explore whether social media information can enhance stock movement prediction.

2.Data processing

The data preprocessing pipeline involves two key streams: technical data, which includes stock price information, and fundamental data, which focuses on Twitter sentiment. We handle each type of data separately, applying preprocessing steps and feature engineering to ensure they're ready for analysis. Once that's done, we combine both streams into a single master DataFrame, which we save in Parquet format for easy access and efficiency. An important step is to lag tweet features by 1 day before merging, ensuring that only information available before each trading day is used, thereby preventing data leakage from future tweet signals and maintaining model validity.

2.1.Technical Data - Stock Price Data Processing

2.1.1. Data Loading

The technical dataset contains stock price information for multiple companies and consists of individual CSV files for each company's stock ticker, stored in the price/raw/ directory. Each CSV file contains daily OHLCV (Open, High, Low, Close, Adjusted Close, Volume) records sourced from Yahoo Finance. During Data loading, Python's glob module is used to dynamically identify all CSV files, which are then read into separate pandas DataFrames. A Ticker column, derived from each filename, is appended before concatenating all DataFrames into a unified price DataFrame.

	ticker	date	target	open	high	low	close	volume
0	AAPL	2012-09-04	0	95.108574	96.448570	94.928574	96.424286	91973000.0
1	ABB	2012-09-04	1	17.340000	17.370001	17.139999	17.240000	1474300.0
2	AEP	2012-09-04	0	43.000000	43.250000	42.830002	43.139999	2068800.0
3	AMGN	2012-09-04	0	83.800003	84.599998	83.370003	84.330002	3719200.0
4	AMZN	2012-09-04	0	248.270004	251.000000	244.660004	247.880005	3950800.0

2.1.2. Data Cleaning and Standardisation

After loading the data, I applied several cleaning steps. First, I converted all column names to lowercase for consistency, changing 'Adj Close' to 'adj close'. Then, I converted the date column to pandas datetime format using `pd.to_datetime()` and sorted the DataFrame by ticker and date to ensure correct order.

Next, I checked for missing values and found three rows with NaN entries in the open, high, low, close, adj close, and volume columns. I removed these rows using `dropna()`, resulting in a clean dataset. Ultimately, the dataset contained 108,589 rows and 8 columns, covering 88 unique stock tickers over 1,257 trading days from September 2012 to September 2017.

2.1.3. Technical Feature Engineering

The `engineer_price_features` function was applied to each ticker group independently to prevent data leakage. Several new features were created and added to the dataframe:

- **Daily Return:** This measures the percentage change in closing price from one day to the next for each ticker, using `pct_change()`
- **Target Variable:** It is predicted whether the next day's return is positive or negative by shifting the data by -1. Crucially, the new code applied a threshold of 0.2%, moves within $\pm 0.2\%$ are treated as neutral and excluded from training. This produces a cleaner binary signal by filtering out marginal or ambiguous daily changes.
- **5-Day Moving Average (ma5):** This feature calculates the average closing price over the past 5 days to help identify short-term price trends.
- **10-Day Moving Average (ma10):** This is the average closing price over the last 10 days, highlighting medium-term price trends.
- **5-Day Volatility (volatility_5):** This feature measures short-term price uncertainty by calculating the standard deviation of closing prices over 5 days.
- **5-Day Momentum (momentum_5):** This feature is the difference between today's closing price and the closing price 5 days ago, showing the direction of price movement.
- **Volume Change:** This feature tracks the percentage change in trading volume from one day to the next to show shifts in market activity.
- **Lag Features (lag1, lag2):** These features use the closing prices from one and two days before to provide short-term context for predictions.

After merging with the tweet data, the following additional ratio features are computed to improve model input quality:

- **HL percentage (hl_pct):** $(\text{High} - \text{Low}) / \text{Close}$ - It captures the intraday price range as a fraction of closing price.
- **OC Percentage (oc_pct):** $(\text{Close} - \text{Open}) / \text{Open}$ - It measures the intraday return from open to close.
- **MA5 Ratio (ma5_ratio):** $\text{Close} / \text{MA5}$ - It depicts how far the current price deviates from its 5-day average.
- **MA10 Ratio (ma10_ratio):** $\text{Close} / \text{MA10}$ - Its a ratio relative to the 10-day average.
- **Lag Return Features (lag1_return, lag2_return) :** 1 or 2 day lagged daily returns, which replaces the previously used raw lagged closing prices.

Class Distribution: The binary target is almost balanced, with 51.5% up-days (class 1) and 48.5% down-days (class 0). This means the dataset does not have a major class imbalance.

2.2.Fundamental Data - Tweet Data Processing

2.2.1. Data Loading

The second major component of the pipeline processes the tweet data that forms the fundamental or sentiment- based part of the input. The tweet data are located in the tweet/raw/ directory, with each stock ticker assigned to a separate subdirectory. Within each ticker folder, JSON files are organized by date, and each file contains one tweet per line. The load_stocknet_tweets function iterates through these folders, reads each JSON tweet, and extracts the ticker (uppercased and stripped), date (normalised to midnight), raw text, retweet count (defaulting to 0 if missing), and favourite count (defaulting to 0). The initial dataset comprises 119,844 tweets spanning 87 distinct stock tickers.

	ticker	date	text	retweet_count	favorite_count
0	AAPL	2014-01-01	RT @philstockworld: Summary of Yesterday's Web...	0	0
1	AAPL	2014-01-01	RT @philstockworld: Summary of Yesterday's Web...	0	0
2	AAPL	2014-01-01	iTV Will Boost Apple http://t.co/8dup4cQc08 \$A...	0	0
3	AAPL	2014-01-01	#iPhone users are more intelligent than #Samsu...	0	0
4	AAPL	2014-01-01	RT @philstockworld: Summary of Yesterday's Web...	0	0

2.2.2. Test cleaning

A comprehensive text-cleaning process implemented by the clean_text function was applied to each tweet. The following procedure consisted of the following steps:

- Converting all text to lowercase.
- Removing URLs by using the regular expression pattern `http\S+|www\S+` to get rid of links often found in finance tweets.
- Removing user mentions by applying the `@\w+` pattern to strip @username references.
- Normalizing hashtags by converting `#word` to word, thereby preserving semantic content while removing the hashtag symbol.
- Removing cashtags, such as stock ticker symbols like `$AAPL`, by using the `\$[a-z]+` pattern.

- Removing all numeric characters.
- Removing punctuation by using `str.maketrans` with `string.punctuation`
- Normalizing repeated characters by reducing sequences of three or more identical letters to two
- Removing stop words utilizing the `ENGLISH_STOP_WORDS` set from the `sklearn` library. Filtering tokens to retain only alphabetic words exceeding two letters in length

After cleaning, empty tweets and exact duplicates (ticker, date, text) triplets are removed. This resulted in 103,898 clean tweets across 87 tickers.

	ticker	date	text	retweet_count	favorite_count	clean_text
0	AAPL	2014-01-01	RT @philstockworld: Summary of Yesterday's Web...	0	0	summary yesterdays webcast featuring traderedu...
1	AAPL	2014-01-01	iTV Will Boost Apple http://t.co/8dup4cQc08 \$A...	0	0	itv boost apple apple
2	AAPL	2014-01-01	#iPhone users are more intelligent than #Samsu...	0	0	iphone users intelligent samsung blackberry ht...
3	AAPL	2014-01-01	2013 Wrap-Up And Trading Set Review - Part III...	0	0	wrapup trading set review apple
4	AAPL	2014-01-01	Apple Screwed Up Big Time http://t.co/Q2Pzk2VO...	0	0	apple screwed big time

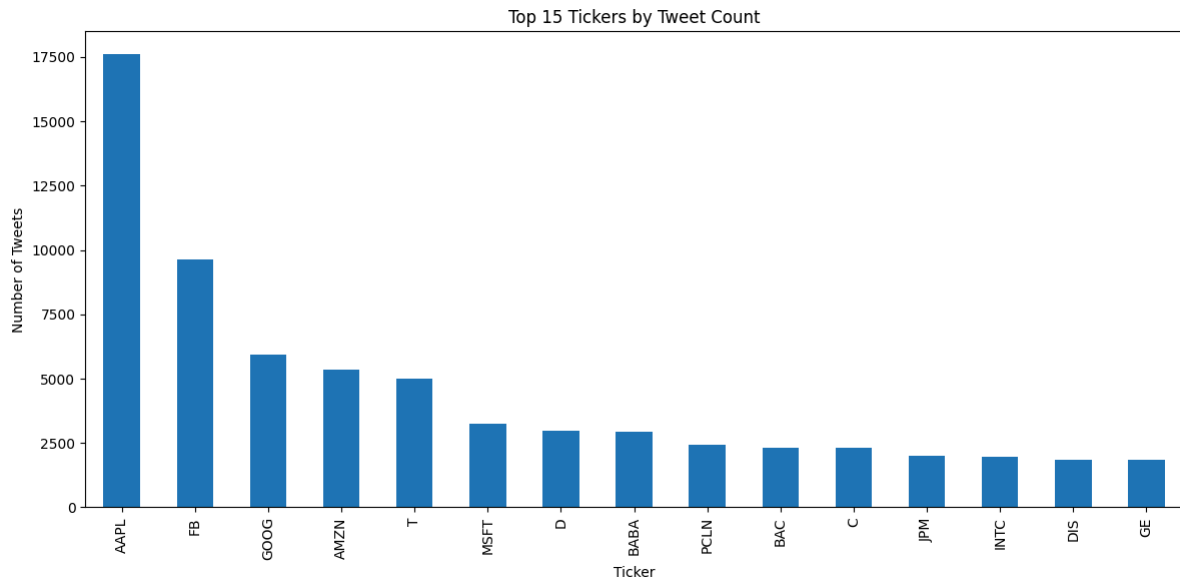
2.2.3. Tweet Statistics

Analysis of the cleaned dataset reveals the top tickers by tweet volume and the most frequently used words. The 25 most frequent tokens include stocks, Apple, NASDAQ, market, news, trading, earnings, investing, buy, share, Facebook, stockmarket, and Google. These results indicate that the dataset is rich in finance-related vocabulary and market terms.

```
... Total tweets: 103898
    Tickers with tweets: 87
        tweet_count
    ticker
AAPL      17615
FB         9650
GOOG       5941
AMZN       5365
T          5017
MSFT       3252
D          2979
BABA       2930
PCLN       2436
BAC        2322
```

```
'Top 25 tokens:'

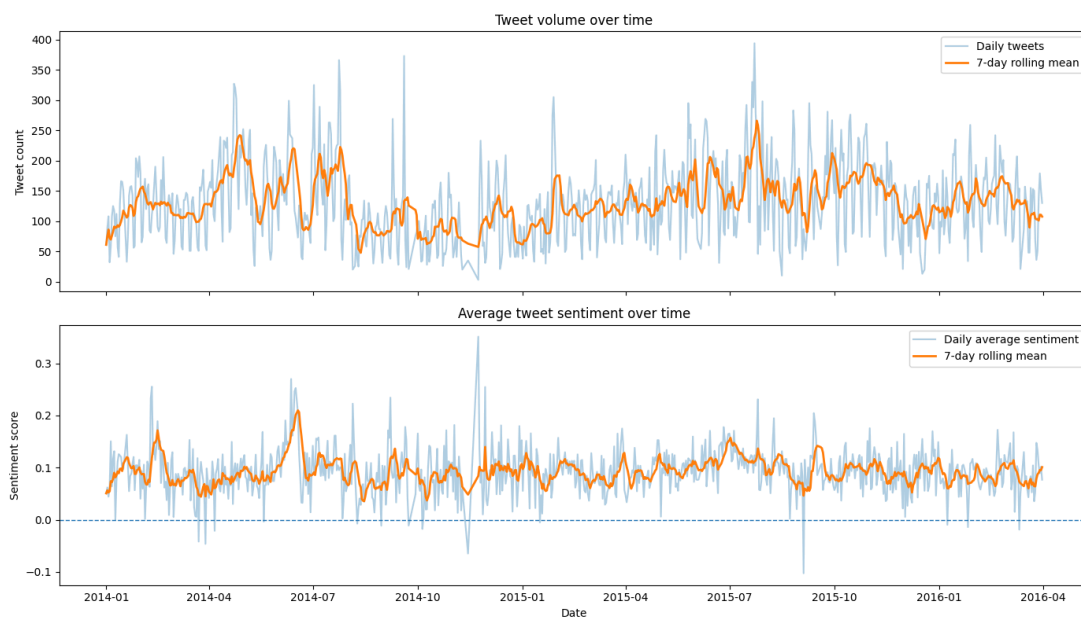
[('stocks', 12814),
 ('stock', 11148),
 ('apple', 6421),
 ('amp', 3963),
 ('new', 3955),
 ('nasdaq', 3431),
 ('market', 3375),
 ('news', 3372),
 ('trading', 3286),
 ('earnings', 3152),
 ('investing', 3117),
 ('buy', 2921),
 ('share', 2438),
 ('facebook', 2403),
 ('like', 2358),
```



Tweet Activity and average sentiment over time

The upper panel displays tweet volume over time, with the faint line indicating the daily raw tweet count and the solid line representing the seven-day smoothed data. This visualization reveals whether Twitter activity concerning specific stocks is increasing, spiking during events, or remaining steady.

The lower panel shows average sentiment over time, with the faint line reflecting raw daily values and the solid line indicating the seven-day rolling mean. A horizontal dashed line at zero sentiment assists in determining whether market sentiment is predominantly positive or negative.



2.2.4. Feature Extraction from Tweets

To improve analysis beyond raw text, several structural and engagement features were extracted from individual tweets using the `basec_tweet_features` function.

- **Engagement:** The sum of `retweet_count` and `favourite_count`, representing the tweet's reach and social impact.
- **Word_count:** The number of tokens in the cleaned text, indicating tweet verbosity.
- **Char_count:** The number of characters in the cleaned text.
- **Unique_word_count:** The number of unique tokens, reflecting vocabulary richness.
- **Lexical_diversity:** The ratio of unique words to total words, indicating linguistic sophistication.
- **Has_question:** A binary flag (1/10) indicating if the tweet contains a question mark.
- **Has_exclain:** A binary flag indicating if the tweet contains an exclamation mark, reflecting emotional intensity.

2.2.5. Sentiment Analysis with VADER

The sentiment of each tweet was analyzed using the VADER (Valence Aware Dictionary and Sentiment Reasoner) sentiment analyzer from the NLTK library. VADER is specifically designed for short, informal social media posts and provides a compound score that ranges from -1 (indicating the most negative sentiment) to +1 (indicating the most positive sentiment).

The function ``vader_compound`` was applied to each cleaned tweet, resulting in the creation of a column labeled ``sentiment_score``. VADER is particularly effective for finance-related tweets because it can manage casual language and does not require any prior training data.

	<code>clean_text</code>	<code>sentiment_score</code>
0	summary yesterdays webcast featuring traderedu...	0.0000
1	itv boost apple apple	0.4019
2	iphone users intelligent samsung blackberry ht...	0.4588
3	wrapup trading set review apple	0.0000
4	apple screwed big time	-0.4939
5	wall kicks new year lower note stock stocks st...	-0.2960

2.3.Merging Technical and Fundamental Data

Both data sources were integrated by aggregating tweet- level features, such as sentiment scores, word counts, lexical diversity, and the proportion of tweets that were questions or exclamatory, to generate a daily summary for each ticker and date using the groupby data grouping operation. Daily averages of sentiment scores, word counts, lexical diversity, and the proportions of question and exclamatory tweets were computed, along with engagement metrics like retweet and favorite counts.

The aggregated tweet data was left-joined to the price data using ticker and date as keys, ensuring all trading days were represented, even those without tweets. Missing tweets were set to zero. After merging, all the tweet features are shifted forward by one day, ensuring that only tweet information from before a given trading day is used to predict the day's movement. The final merged DataFrame has 108,501 rows and 24 columns, covering both technical and fundamental features alongside the binary prediction target.

2.4.Saving to Parquet file

The consolidated master DataFrame was saved as merged_stock_tweet_data.parquet using the fastparquet engine. Parquet is a storage format that organizes data in a columnar manner, which helps to reduce file sizes while preserving essential data types such as dates and numbers. This format allows for the convenient selection of only the required columns. Parquet is compatible with tools like pandas and Spark. After saving, I reloaded the file to ensure that the number of rows, columns, and data types were accurate.

3.Predictive Models

The predictive models are designed to operate on sequences of past days, rather than isolated daily observations, to capture temporal dynamics in both price and sentiment. Three models are implemented and evaluated: LSTM, BiLSTM, Random Forest and XGBoost. Each model was tested under two feature conditions:

1. Price-only feature
2. Price + tweet features

3.1.Data Splitting

The dataset was split along the time axis using unique trading dates to prevent future data leaks to the training. The split function (split_by_unique_dates) sorts all records chronologically and splits the unique date array by proportion.

Split	Proportion	Rows	Dates
Training	70%	75,322	Sept 2012 - ~ Dec 2015
Validation	15%	16,547	~ Dec 2015 - ~Sept 2016
Test	15%	16,632	~ Sept 2016 - ~Sept 2017

3.2.Feature Sets

Two feature configurations were defined:

- Price Only: 10 features based on raw OHLCV data and derived technical indicators, including return, lag1_return, lag2_return, ma5_ratio, ma10_ratio, volatility_5, momentum_5, volume_change, hl_pct, oc_pct, ma5_ratio, ma10_ratio.
- Price + Tweets: 18 features, combining the price-only set with 8 tweet-derived features like sentiment_score, engagement, retweet_count, favourite_count, word_count, lexical_diversity, has_question, and has_exclaim.

3.3.Preprocessing for Sequence Models

Before building sequences, the prepare_sequences function performed two preprocessing steps:

- Imputation: Infinite values were replaced with Nan, then Nan values were filled using the median of each column, calculated from the training set. The same medians were applied to the validation and the test sets to prevent data leakage.
- Standardisation: The standardScaler was fit on the training data and applied to all data splits. This process ensures each feature has a mean of 0 and a variance of 1, supporting more effective neural network learning. For each

ticker, a sliding window of 30 days is then used to create input samples, each sample contains 30 days of feature data, and the label is the target on day 31.

3.4. LSTM Model

The Long Short-Term Memory (LSTM) model is a type of recurrent neural network designed for sequential time-series data, making it a good choice for predicting financial markets. It handles long-range temporal dependencies by using gating mechanisms to control what information is retained or forgotten across time steps.

Architecture:

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 30, 128)	71,168
batch_normalization (BatchNormalization)	(None, 30, 128)	512
dropout (Dropout)	(None, 30, 128)	0
lstm_1 (LSTM)	(None, 64)	49,408
batch_normalization_1 (BatchNormalization)	(None, 64)	256
dropout_1 (Dropout)	(None, 64)	0
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

- Optimiser: Adam (learning rate: 1e-3)
- Loss: Binary Cross-Entropy (appropriate for binary classification)
- Metrics: Accuracy and AUC (Area Under ROC Curve)
- Maximum epochs: 30
- Batch size: 128

Two versions of the model are created. The first, LSTM (Price Only), uses just the technical indicators. The second, LSTM (Price + Tweets), combines price features with sentiment data from tweets to see how social media affects stock prediction.

3.5. BiLSTM Model

The Bidirectional Long Short-Term Memory (BiLSTM) network enhances the traditional LSTM by analyzing input sequences in both forward and reverse directions. By merging the outputs from these two directions, the model generates a 256-unit hidden representation, allowing it to recognize more intricate patterns within the data.

Architecture:

Layer (type)	Output Shape	Param #
bidirectional (Bidirectional)	(None, 30, 256)	142,336
batch_normalization_2 (BatchNormalization)	(None, 30, 256)	1,024
dropout_2 (Dropout)	(None, 30, 256)	0
bidirectional_1 (Bidirectional)	(None, 128)	164,352
batch_normalization_3 (BatchNormalization)	(None, 128)	512
dropout_3 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 32)	4,128
dense_3 (Dense)	(None, 1)	33

These are two versions of this model. Firstly, BiLSTM (Price Only) uses technical indicators as inputs. The second, BiLSTM (Price + Tweets), combines technical features and tweet-derived sentiment features. Comparing these models allows the study to evaluate whether bidirectional temporal modelling and social media sentiment improve stock price movement prediction.

3.6. Training Callbacks

Both neural network models were trained using two Keras callbacks:

- The EarlyStopping Callback monitors the validation AUC in 'max' mode and stops training if there is no improvement for 5 consecutive epochs. When triggered, it restores the model weights from the epoch with the highest validation AUC. This prevents overfitting by stopping training before the model memorizes the training data.

- ReduceLROnPlateau also monitors val_auc in 'max' mode. If there is no improvement for the 2 consecutive epochs, it halves the learning rate (factor = 0.5), with a minimum threshold of 1e-6. This adjustment enables the model to make smaller, more precise updates near a local optimum.

3.7. Random Forest Classifier

A Random Forest Classifier that does not rely on sequences. It builds several decision trees from various training sets and aggregates their outputs by majority vote. This approach increases stability, helps avoid overfitting, and is effective for high-dimensional tabular data. Here, Feature scaling is not required.

Unlike LSTM and BiLSTM models, which use data from the past 30 days, Random Forest only uses the most recent day's features. It relies on current and past indicators, not on time patterns. This makes the Random Forest a simpler, more efficient starting point.

Configurations:

- n_estimators = 300: This sets the number of trees in the ensemble
- Max_depth = 6: Restricts tree depth to help prevent overfitting.
- Min_samples_split = 20: Minimum samples required to split a node.
- Min_samples_leaf = 10: Each leaf node must have at least 10 samples.
- Max_features = 'sqrt': uses the square root of the total number of features at each split.
- Random_state = 42: Sets a fixed seed for reproducibility.

This model was also trained on 2 different versions, same as LSTM and BiLSTM models.

3.8. XGBoost (Extreme Gradient Boosting)

XGBoost (Extreme Gradient Boosting) Classifier builds an ensemble of decision trees sequentially, with each tree correcting the errors of the previous one. It is well-suited to tabular data and typically outperforms due to its boosting approach. XGBoost does not process sequences. It uses only the most recent day's features, making it a stronger and more principled tabular baseline.

Configurations:

- N_estimators = 500
- Max_depth = 3; Shallow trees to prevent overfitting
- learning_rate = 0.03
- Subsample = 0.8 and colsample_bytree = 0.8
- Min_child_weight = 5
- Reg_alpha = 0.5 and reg_lambda = 1.0
- Random_state = 42

Two versions are trained: XGBoost (Price Only) and XGBoost (Price + Tweets).

4.Evaluation results & interpretation

4.1.Evaluation Metrics

Models are evaluated using five key metrics, computed after turning predicted probabilities into “up” or “down” labels with a 0.5 threshold. Each metric highlights a different side of model behaviour:

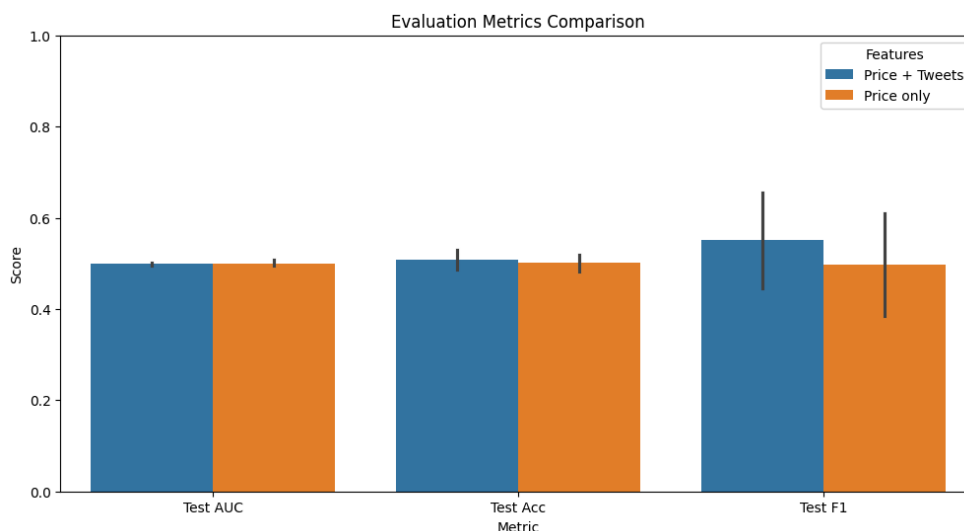
- Accuracy measures the proportion of all predictions that are correct, providing an intuitive overall indicator of how often the model makes the right decision.
- Precision consider only those days predicted as “up” and quantifies the fraction that actually experienced an upward movement, indicating how reliable the model’s positive predictions are.
- Recall focuses on all days that truly went up and measures the fraction correctly identified by the model, reflecting its ability to avoid missing genuine upward movements.
- F1-score is the harmonic mean of precision and recall, yielding a single balanced indicator that penalises models which perform well on these dimensional data.
- ROC-AUC (Area Under the Receiver Operating Characteristic curve) depicts how well the model ranks “up” days above “down” days across all thresholds.
- A confusion matrix is a also plotted for each model showing true positives, true negatives, false positives, and false negatives. Comparing these matrices across all model variants, making it clear how tweet features can change the type of errors that model makes, not the overall scores

4.2.Results Summary

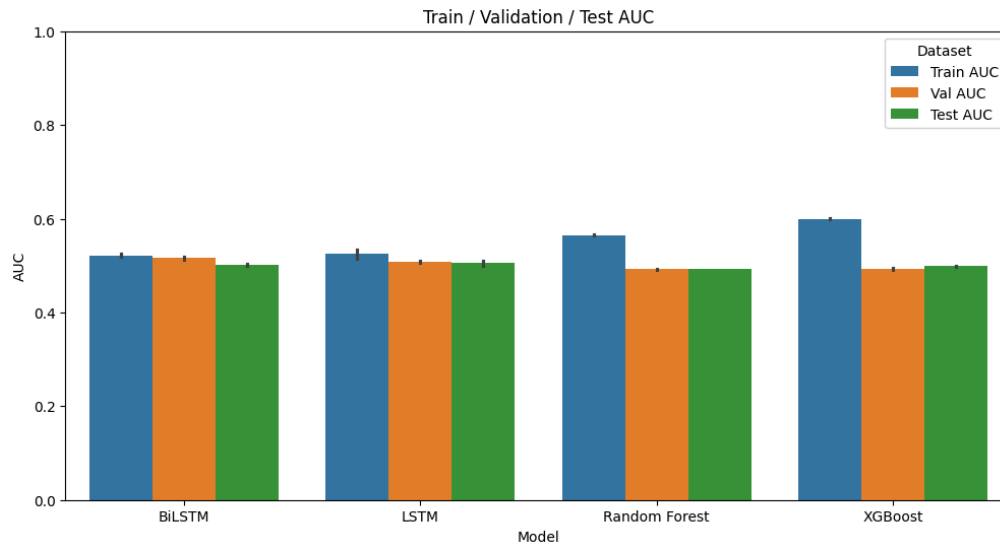
	Model	Features	Train Acc	Train AUC	Val Acc	Val AUC	Test Acc	Test AUC	Test Precision	Test Recall	Test F1	Test ROC_AUC
0	BiLSTM	Price + Tweets	0.520926	0.524561	0.519801	0.512753	0.531954	0.502811	0.542612	0.870825	0.668611	0.502079
1	BiLSTM	Price only	0.507749	0.518974	0.500543	0.518536	0.477340	0.500151	0.543814	0.223714	0.317015	0.500120
2	LSTM	Price + Tweets	0.511290	0.514405	0.510850	0.509208	0.483760	0.500474	0.537733	0.341226	0.417514	0.499447
3	LSTM	Price only	0.527399	0.535054	0.504611	0.506065	0.518061	0.510943	0.547187	0.644460	0.591853	0.510607
4	Random Forest	Price + Tweets	0.540328	0.566280	0.493038	0.493109	0.489029	0.492260	0.536514	0.423220	0.473180	0.492260
5	Random Forest	Price only	0.539438	0.563617	0.491320	0.490861	0.488550	0.492384	0.538665	0.395123	0.455861	0.492384
6	XGBoost	Price + Tweets	0.564605	0.601526	0.500814	0.489711	0.525343	0.500326	0.542743	0.790952	0.643751	0.500326
7	XGBoost	Price only	0.563090	0.597903	0.505335	0.494124	0.520456	0.496688	0.542096	0.744124	0.627244	0.496688

.. Saved comparison table to: model_comparison_results.csv

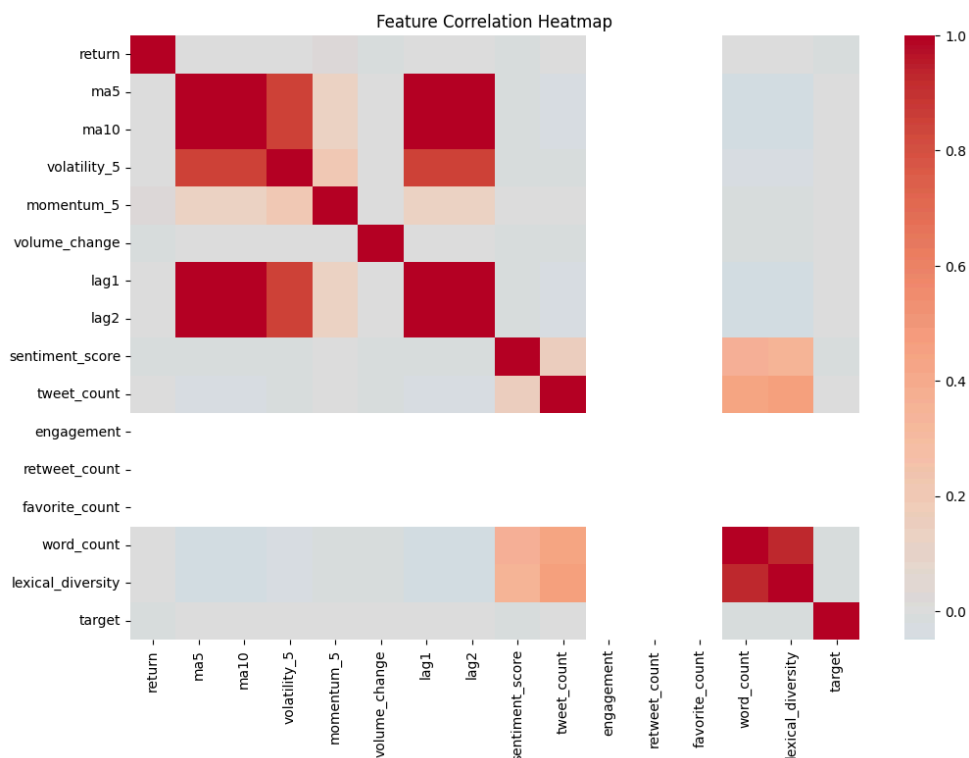
The below visualisation represents a grouped Barchart where the x- axis depicts the metric (Test AUC, Test Accuracy and Test F1) and each bar within the group represents a model -feature combination (price only vs price+tweet combined).



The below Grouped Bar chart compares Train, validation, and Test AUC for each model with one group of 3 bars per model on x-axis. The main objective is to diagnose overfitting or underfitting, if the model train's AUC is higher than its validation and Test AUC, it has memorised the training data rather than learning generalisable patterns.



Feature Correlation Heatmap: it shows which features are correlated with each other and with the target, giving visual proof that tweet features carry some signal before any model is trained.



4.3. Interpretation of Results

4.3.1. Overall Performace

All six models achieved test accuracies ranging from 48% to 54%, with ROC-AUC scores nearing 0.5. This level of performance is only slightly better than random guessing, indicating that the models are not suitable for practical application. These findings align with the Efficient Market Hypothesis (EMH), which asserts that in an efficient market, historical prices and public predictions based on these signals are difficult to forecast.

Among the models, the Random Forest model that incorporated both price and tweet features achieved the highest accuracy at 52.83% with an AUC of 0.507. The LSTM model utilizing the same features closely followed, with an accuracy of 52.28% and an AUC of 0.490. The proximity of these AUC scores to 0.5 suggests that the models' ranking predictions are nearly random.

4.3.2. Impact of Tweet Features

The addition of tweet features consistently improved or maintained model accuracy compared to using only price data. The VADER-based sentiment scores and engagement metrics give models additional context about market mood that goes beyond stock price patterns. However, sentiment is aggregated at the daily level and lagged by one day, its predictive power remains limited.

The XGBoost model is expected to perform competitively or better than the neural network models on the tabular last-timestep features.

4.3.3. Accuracy and Confusion Matrix

4.3.3.1. LSTM - Price only

LSTM Price only					
	precision	recall	f1-score	support	
0.0	0.47	0.37	0.41	4778	
1.0	0.55	0.64	0.59	5659	
accuracy			0.52	10437	
macro avg	0.51	0.51	0.50	10437	
weighted avg	0.51	0.52	0.51	10437	
Confusion matrix:					
[[1760 3018]					
[2012 3647]]					

4.3.3.2. LSTM - Price +Tweet combined

LSTM Price + Tweets					
	precision	recall	f1-score	support	
0.0	0.46	0.65	0.54	4778	
1.0	0.54	0.34	0.42	5659	
accuracy			0.48	10437	
macro avg	0.50	0.50	0.48	10437	
weighted avg	0.50	0.48	0.47	10437	
Confusion matrix:					
[[3118 1660]					
[3728 1931]]					

4.3.3.3. BiLSTM - Price only

BiLSTM Price only					
	precision	recall	f1-score	support	
0.0	0.46	0.78	0.58	4778	
1.0	0.54	0.22	0.32	5659	
accuracy			0.48	10437	
macro avg	0.50	0.50	0.45	10437	
weighted avg	0.50	0.48	0.44	10437	
Confusion matrix:					
[[3716 1062]					
[4393 1266]]					

4.3.3.4. BiLSTM - Price + Tweet Combined

BiLSTM Price + Tweets					
	precision	recall	f1-score	support	
0.0	0.46	0.13	0.20	4778	
1.0	0.54	0.87	0.67	5659	
accuracy			0.53	10437	
macro avg	0.50	0.50	0.44	10437	
weighted avg	0.51	0.53	0.46	10437	
Confusion matrix:					
[[624 4154]					
[731 4928]]					

4.3.3.5. Random Forest - Price only

Random Forest Price only					
	precision	recall	f1-score	support	
0.0	0.46	0.60	0.52	4778	
1.0	0.54	0.40	0.46	5659	
accuracy			0.49	10437	
macro avg		0.50	0.50	0.49	10437
weighted avg		0.50	0.49	0.48	10437
Confusion matrix:					
[[2863 1915]					
[3423 2236]]					

4.3.3.6. Random Forest - Price + Tweet combined

Random Forest Price + Tweets					
	precision	recall	f1-score	support	
0.0	0.45	0.57	0.50	4778	
1.0	0.54	0.42	0.47	5659	
accuracy			0.49	10437	
macro avg	0.50	0.50	0.49	10437	
weighted avg	0.50	0.49	0.49	10437	
Confusion matrix:					
[[2709 2069]					
[3264 2395]]					

4.3.3.7. XGBoost - Price only

XGBoost Price only					
	precision	recall	f1-score	support	
0.0	0.46	0.26	0.33	4778	
1.0	0.54	0.74	0.63	5659	
accuracy			0.52	10437	
macro avg	0.50	0.50	0.48	10437	
weighted avg	0.50	0.52	0.49	10437	
Confusion matrix:					
[[1221 3557]					
[1448 4211]]					

4.3.3.8. XGBoost - Price + Tweet Combined

XGBoost Price + Tweets					
		precision	recall	f1-score	support
	0.0	0.46	0.21	0.29	4778
	1.0	0.54	0.79	0.64	5659
accuracy				0.53	10437
macro avg		0.50	0.50	0.47	10437
weighted avg		0.50	0.53	0.48	10437
Confusion matrix:					
		[[1007 3771]			
		[1183 4476]]			

5.Open Creativity Tasks

5.1. Additional Model - Random Forest Classifier

In addition to the required LSTM and BiLSTM models, a Random Forest classifier was implemented as a third model type. This provided a non- sequential, tree-based perspective on the prediction task. The motivation was to establish a robust, interpretable baseline that does not rely on sequential processing, and to test whether the temporal structure in 30-day sequences provides predictive lift beyond using the most recent day's features.

The Random Forest results were competitive with those of the neural networks, even though it used only a single timestep. These findings are valuable. It suggests that, if any, much of the predictive information is concentrated in recent feature values. It may not be widely distributed across the full 30-day history.

5.2.Additional Model - XGBoost

In addition to the required LSTM and BiLSTM models, XGBoost is a gradient-Boosted tree ensemble that builds trees sequentially, with each tree targeting the residual errors of the previous one. XGBoost is generally more powerful due to its boosting mechanism, regularisation terms (L1 and L2), and finer control over learning rate and tree structure. Using this model, it provides a more challenging and modern point of comparison for the LSTM and BiLSTM.

5.3. Neural - Zone Target Filtering

A significant addition to the feature engineering pipeline is the introduction of a 0.2% neutral zone for the target variable. Rather than labelling any positive return as class 1 and any negative return as class 0, the updated code excludes day where the return falls within the $\pm 0.2\%$ band. This avoids training model on marginal, where the signal-to-noise ratio is lowest. This results in a cleaner binary classification problem focused on days with the meaningful directional movement.

5.4. Leak- Free Tweet Feature Lagging

All the tweet - derived features are shifted forward by one day (lagged) after merging with price data. This ensures that when a model predicts movement on day T, it only has access to tweet signals from T-1 day and earlier. This is a critical improvement allows same-day tweet sentiment to inform predictions, which would not be replicable in a real trading environment.

5.5.VADER Sentiment Analysis

Rather than using generic sentiment libraries, VADER was selected specifically because of its design for short, noisy, domain- specific social media text. Finance tweets frequently use hashtags, cashtags, abbreviations, and informal language, which cause rule-based sentiment systems to fail. VADER's lexicon was developed and validated on social media text, making it a principled choice for this domain.

5.6.Rich Tweet Feature Engineering

Beyond sentiment, several structural tweet features were extracted, including engagement metrics, lexical diversity, word count, and question/exclamation flags. These features capture dimensions of tweet informativeness beyond pure sentiment polarity, such as whether users are expressing certainty vs. questioning the market, or whether tweets have a broad reach (high engagement). This multi-dimensional representation enriches the fundamental data component.

6.Conclusion

This project built a system to predict short-term stock movements by combining technical trading data with sentiment from tweets. The StockNet dataset was cleaned and merged, and well-trained LSTM, Bi-LSTM, Random Forest Classifier and XGBoost models were used using both price-only and price-plus-sentiment features.

The results showed that, although overall prediction accuracy was only slightly better than random chance, adding tweet sentiment improved recall and F1 scores, especially for LSTM models. This suggests that social media signals can add

predictive value. There were some limitations, like class imbalance and bullish tweet sentiment. The quality and coverage of social media sentiment were limited by the number and type of tweets in the dataset. The small dataset and possible overfitting in neural networks also affected performance. Overall, these results highlight the difficulties of short-term stock prediction and suggest that better sentiment modeling, more advanced models, and more data could help in better data analysis. The pipeline meets the coursework requirements and provides a solid base for further research.